

UNIVERSIDAD
DE GRANADA

E.T.S DE INGENIERIA INFORMÁTICA Y DE TELECOMUNICACIONES

PRÁCTICA EVALUABLE II – ADAPTACIÓN DINÁMICA DE LA WEB DE LA AGENCIA DE VIAJE DE LA PRÁCTICA EVALUABLE I

PROGRAMACIÓN WEB

DOCENTE: JUAN MANUEL FERNÁNDEZ LUNA

Andrés Pozo Chiclana

FECHA DE ENTREGA: 07/06/2026

1. INTRODUCCIÓN

En esta segunda práctica se ha ampliado el sitio web de Nómada 360, incorporando nuevas funcionalidades dinámicas mediante PHP, JavaScript y una base de datos MySQL.

Gracias a PHP se ha podido generar contenido dinámico, reutilizar partes comunes de la página como la cabecera y el pie de página, trabajar con sesiones y procesar formularios. También se ha creado una base de datos con tablas para almacenar usuarios y viajes, permitiendo que el catálogo de destinos se muestre de forma dinámica.

Además, gracias a JavaScript se validan estos formularios antes de enviarlos al servidor y para implementar un carrusel dinámico. Este carrusel obtiene viajes aleatorios desde la base de datos y muestra su imagen, destino y frase descriptiva.

2. EXPLICACIÓN DE LOS FICHEROS PHP

En esta práctica se han utilizado distintos ficheros PHP para convertir la web estática en una web dinámica. Estos ficheros permiten reutilizar partes comunes de la página, conectarse a la base de datos, gestionar sesiones, mostrar viajes y procesar formularios.

Los ficheros PHP se pueden dividir en tres grupos principales: las páginas principales de la web, los ficheros de la carpeta includes y los ficheros de procesamiento de la carpeta actions.

2.1. FICHEROS PRINCIPALES

- index.php

  El fichero index.php corresponde a la página principal. En esta página se muestra la cabecera común, un carrusel dinámico, una sección de bienvenida y un buscador de viajes.

  Al inicio del archivo se inicia la sesión y se carga la conexión a la base de datos. Después se realiza una consulta SQL para obtener varios viajes aleatorios de la tabla “viajes”, la cual permite que el carrusel no sea fijo, sino que muestre destino diferentes obtenidos desde la base de datos.

  Los datos recuperados se guardan en la variable $viajesCarrusel y posteriormente se pasan a JavaScript mediante json_encode(). De esta forma, el archivo carrusel.js puede utilizar esos datos para mostrar la imagen, el destino y la frase de cada viaje.

Además, se muestran mensajes de sesión relacionados con el registro y el inicio de sesión. Por ejemplo, si un usuario se registra correctamente, se guarda un mensaje en $_SESSION[“mensaje_registro”] y se muestra en la página principal. También se muestra un mensaje si el usuario introduce mal sus datos de acceso. Finalmente, esta página incluye el formulario de búsqueda de viajes, el cual envía los datos mediante GET a viajes_buscados.php, permitiendo buscar viajes por destino y/o fecha.

- altausuarios.php

  El fichero altausuarios.php contiene el formulario de registro de nuevos usuarios. En este se solicitan datos personales y datos de acceso, como el nombre, los apellidos, el nombre de usuario, el correo o la contraseña.

  El fichero utiliza POST para enviar los datos al archivo actions/procesarAltaUsuario.php. Además, este formulario se valida con los ficheros JavaScript correspondientes antes de enviarlo.

  También muestra mensajes de sesión mediante $_SESSION[“mensaje_registro”]. Esto permite informar al usuario si el registro se ha completado correctamente o si ha ocurrido algún error, como que el usuario o el correo ya estén registrados.

- viajes.php

  Esta página muestra el catálogo dinámico de viajes. A diferencia de la primera práctica, los viajes ya no están escritos directamente en HTML, sino que se obtienen desde la tabla “viajes” de la base de datos.

  Al principio del archivo se cargan la conexión, includes/conexión.php, y las funciones comunes, includes/funciones.php. Después se obtiene el filtro de país o continente mediante list($where, $params) = filtroViajes(), la cual permite saber si el usuario ha seleccionado un país o continente desde el menú lateral. En función de ese filtro, se cuentan los viajes disponibles y se calcula la paginación.

  La página muestra 9 viajes por página y, a continuación, se calcula el desplazamiento mediante $offset y se obtiene los viajes correspondientes. Finalmente, se muestra una paginación para permitir al usuario ver la información sobre los siguientes viajes.

- viajes_buscados.php

  Este fichero muestra los resultados del buscador de la página principal. Recibe los datos enviados por GET desde el formulario de index.php y después llama a la función buscarViajes($conexion, $destino, $fecha). Esta función busca viajes cuyo destino, país, continente o descripción coincidan con el texto introducido por el usuario. Además, si

se introduce una fecha, se buscan viajes cuya fecha de inicio y fecha de fin incluya dicha fecha.

Si no se encuentran resultados, la página muestra un mensaje indicando que no hay viajes que coincidan con la búsqueda. Si hay resultados, se muestran usando la misma función de tarjetas utilizada en viajes.php, lo cual permite reutilizar código y mantener el mismo diseño visual para todos los listados de viajes.

- gestion_viajes.php

  El fichero gestion_viajes.php es la página de administración de viajes y solo puede acceder a ella un usuario de tipo administrador. Por ello, al inicio del archivo se comprueba si existe una sesión iniciada y si el tipo de usuario es administrador. Esta página funciona según el valor de la variable accion, recibida por GET. Las acciones principales son listar, crear, editar y eliminar viajes.

  Cuando la acción es listar, se ejecuta una consulta para obtener todos los viajes: $sql = “SELECT * FROM viajes ORDER BY destino”;

  Cuando la acción es editar o eliminar, se obtiene el viaje correspondiente mediante la función obtenerViajePorId($conexion, $id).

  Los formularios de crear, editar y eliminar se procesan en archivos dentro de la carpeta actions. De esta forma, gestion_viajes.php se encarga de mostrar la interfaz de administración, mientras que los archivos de actions se encargan de modificar la base de datos.

2.2. FICHEROS AUXILIARES

La carpeta includes contiene ficheros PHP auxiliares que se reutilizan en varias páginas de la web, de forma que se evita repetir código y se organiza mejor cada parte del sitio web.

En esta carpeta se encuentran los siguientes ficheros:

- cabecera.php

  El fichero cabecera.php contiene la cabecera común de la web. Esta cabecera se incluye en las páginas principales mediante require_once, de forma que todas las páginas mantienen un encabezado común.

  Al inicio del fichero se comprueba si la sesión ya está iniciada, de forma que se evita iniciar sesión dos veces y permite utilizar las variables de sesión dentro de la cabecera.

Si el usuario ya ha iniciado sesión, se muestra su nombre de usuario, su tipo de cuenta y un enlace para cerrar sesión.

Si no hay ningún usuario conectado, la cabecera muestra el formulario de login, el cual se envía a actions/procesarLogin.php mediante POST.

La cabecera también incluye el menú de navegación de la web con enlaces al resto de las páginas. Además, si el usuario conectado es de tipo administrador, se muestra una opción adicional que permite acceder a gestion_viajes.php, donde se pueden crear, editar y eliminar viajes.

- pie.php

  Este fichero contiene el pie de página común de la web. Se incluye al final de las páginas principales para mantener una estructura común en todo el sitio.
  En este aparece enlaces como el acceso a la página de contacto y al documento como_se_hizo.pdf y también muestra la fecha de la última modificación de la web.

- conexion.php

  El fichero conexion.php se encarga de establecer la conexión con la base de datos de Bahia. Para ello, se definen los datos necesarios para conectarse: $user, $password, $host y $database. A partir de estos datos se construye la cadena de conexión (DSN) y después se crea un nuevo objeto PDO.

  También se configura el modo de errores para que PDO lance excepciones si ocurre algún problema con la base de datos.

- funciones.php

  Este último fichero contiene funciones reutilizables relacionadas con usuario, viajes, búsquedas, menús y paginación, de forma que se evita repetir código en distintas páginas. Las funciones definidas en este fichero son:

  - existeUsuarioOCorreo: esta función comprueba si ya existe un usuario registrado con el mismo nombre de usuario o correo electrónico, de forma que se evitan duplicados en la base de datos.
  - insertarUsuario: esta función inserta un nuevo usuario en la tabla “usuario”. Antes de guardar la contraseña, se cifra mediante password_hash(), de forma que no se almacena en texto plano en la base de datos. El tipo de usuario insertado siempre es cliente, ya que los registros realizados desde la web corresponden a usuarios normales.
  - obtenerDestinosMenu: obtiene los continentes y países existentes en la tabla “viajes”. A partir de esta información se genera el menú lateral dinámico.

- mostrarMenuLateralViajes: muestra el menú lateral en la página de viajes en la página de resultados de búsqueda. Además, permite filtrar los viajes por país o por continente.
- filtroViajes: comprueba si se ha seleccionado un país o continente.
- contarViajes: calcula cuántos viajes cumplen el filtro actual o cuántos viajes hay disponibles en caso de que no se seleccione ningún filtro.
- obtenerViajesPaginados: obtiene los viajes que deben mostrarse en la página actual.
- mostrarTarjetaViaje: se encarga de mostrar la información de un viaje en formato de tarjeta. Esta tarjeta incluye la imagen, el destino, la descripción, el alojamiento, las fechas, el precio y la duración. Además, si el usuario conectado es administrador, muestra los enlaces para editar y eliminar viaje.
- mostrarPaginacion: esta función se encarga de mostrar los botones de navegación entre páginas de viajes y de limitar la navegación del usuario entre la primera y la última página de viajes, evitando que el usuario intente ir a páginas inexistentes.
- obtenerViajePorId: permite obtener los datos de un viaje concreto a partir de su identificador.
- buscarViajes: permite buscar viajes según el texto introducido por el usuario y/o una fecha concreta.

2.3. FICHEROS DE PROCESAMIENTO

La carpeta actions contiene los ficheros PHP encargados de procesar los formularios de la web y el logout. A diferencia de las páginas principales, estos archivos no se utilizan para mostrar contenido visual al usuario, sino para recibir datos, realizar operaciones sobre la base de datos y redirigir a otra página.

- procesarAltaUsuario.php

  El fichero procesarAltaUsuario.php procesa el formulario de registro de nuevos usuarios. Primero inicia la sesión y carga los ficheros necesarios: session_start(), require_once "../includes/conexión.php", require_once "../includes/funciones.php". Se utiliza ../ porque este archivo se encuentra dentro de la carpeta actions, por lo que debe subir un nivel para acceder a la carpeta includes.

Después se comprueba que el formulario haya sido enviado mediante el método POST, de esta forma, si alguien intenta acceder al archivo sin enviar el formulario, se le redirige de nuevo a la página de registro.

A continuación, se recogen los datos enviados desde el formulario, como el nombre, apellidos, fecha de nacimiento, teléfono, correo, usuario y contraseña. También se obtiene la confirmación de contraseña para comprobar que ambas coinciden.

El archivo comprueba si hay campos vacíos y si las contraseñas coinciden. Aunque la validación principal se realiza con JavaScript, estas comprobaciones sirven como protección adicional.

Después se comprueba si ya existe un usuario con el mismo nombre de usuario o correo electrónico mediante la función existeUsuarioOCorreo. Si el usuario o el correo ya existen, se guarda un mensaje de error en la sesión y se redirige a altausuario.php.

Si no hay duplicados, se llama a la función insertarUsuario, que inserta el nuevo usuario en la base de datos. Además, antes de guardar la contraseña, esta se cifra mediante password_hash(), lo que evita almacenarla en texto plano.

Finalmente, si el registro se completa correctamente, se guarda un mensaje de éxito en la sesión y se redirige al usuario a la página principal para que pueda iniciar sesión.

- procesarLogin.php

  El fichero procesarLogin.php procesa el formulario de inicio de sesión que aparece en la cabecera de la web.

  Primero inicia sesión y carga la conexión a la base de datos. Después comprueba si el formulario se ha enviado mediante POST, si el usuario o la contraseña están vacíos, redirige a la página principal.

  Para comprobar el inicio de sesión, se busca primero el usuario en la base de datos. Una vez recuperado el usuario, se utiliza la función password_verify() para comprobar si la contraseña introducida coincide con la contraseña cifrada almacenada.

  Si la comprobación es correcta, se guardan en la sesión los datos principales del usuario: id_usuario, usuario y tipo. Gracias a estas variables de sesión, el resto de páginas pueden saber si hay un usuario conectado y si es cliente o administrador.

  Si el usuario o la contraseña son incorrecto, se guarda un mensaje de error en la sesión y, finalmente, el archivo redirige siempre a index.php.

- logout.php

  El fichero logout.php se encarga de cerrar la sesión del usuario. Para ello, primero se

inicia la sesión y después se eliminan todas las variables de sesión con session_unset() y session_destroy(). Finalmente, se redirige al usuario a la página principal.

- procesarCrearViaje.php

  Este fichero procesa el formulario de creación de viajes desde la página gestion_viajes.php.

  Antes de realizar ninguna operación, comprueba que el usuario conectado sea administrador, de esta forma, se impide que un usuario normal pueda crear viajes aunque intente acceder directamente al archivo.

  Una vez enviado el formulario, se prepara una sentencia INSERT para insertar un nuevo viaje en la tabla "viajes". Los datos insertados son el destino, país, continente, fechas, precio, duración, imagen, frase para el carrusel, descripción y alojamiento.

  La consulta utiliza parámetros preparados y cada parámetro se rellena con los datos recibidos del formulario. Al finalizar la inserción, se redirige al administrador al listado de viajes. De esta manera, el nuevo viaje aparece automáticamente en la gestión de viajes y también en la parte pública de la web.

- procesarEditarViaje.php

  El fichero procesarEditarViaje.php procesa el formulario de edición de un viaje existente. Al igual que en la creación de viajes, primero se comprueba que el usuario conectado sea administrador. Después, se ejecuta una sentencia UPDATE sobre la tabla "viajes".

  El formulario de edición incluye un campo oculto, type="hidden", con el identificador del viaje, de forma que se sabe qué viaje se debe modificar.

  La sentencia SQL actualiza todos los campos editables del viaje y, después, el archivo redirige de nuevo al listado de viajes de la zona de administración.

- procesarEliminarViaje.php

  Este fichero procesa la eliminación de un viaje. Este archivo también está protegido para que solo pueda ejecutarlo un usuario administrador. Si el usuario no está conectado o no es administrador. Si el usuario no está conectado o no es administrador, se redirige a la página principal.

  Cuando se recibe el formulario, se obtiene el identificador del viaje y después se ejecuta una sentencia DELETE. Al finalizar, se redirige al administrador al listado de viajes.

3. EXPLICACIÓN DE LOS FICHEROS JAVASCRIPT

Los ficheros JavaScript se han empleado para validar formularios antes de enviarlos al servidor y para crear un carrusel dinámico en la página principal. Estos ficheros se encuentran en la carpeta js y cada uno tiene una función concreta.

- carrusel.js

  El fichero carrusel.js controla el carrusel dinámico de la página principal. En index.php se realiza una consulta a la base de datos para obtener varios viajes aleatorios. Después, esos datos se pasan a JavaScript mediante json_encode() y se guardan en la variable viajesCarrusel.

  El archivo carrusel.js utiliza esa variable para mostrar en pantalla la imagen, el destino y la frase de cada viaje. Primero se comprueba que la variable viajesCarrusel exista y que contenga datos. Después se inicializa la variable posicionActual, que indica qué viaje se está mostrando en ese momento.

  La función mostrarViaje obtiene el viaje situado en la posición actual del array y actualiza los elementos HTML del carrusel.

  Además, el carrusel cuenta con dos botones: una flecha izquierda y una flecha derecha. Al pulsar la flecha derecha, se avanza al siguiente viaje. Si se llega al final del array, se vuelve al primer viaje. Al pulsar la flecha izquierda, se retrocede al viaje anterior. Si se está en el primer viaje, se pasa al último.

- validacionesComunes.js

  El fichero validacionesComunes.js contiene funciones generales que se reutilizan en los distintos formularios de la web. Las funciones principales son:

  - estaVacio: esta función comprueba si un campo está vacío y elimina los espacios en blanco.
  - numeroPositivo: esta función comprueba si un valor es numérico y mayor que cero. Se utiliza en campos como el precio y la duración.
  - fechaValida: comprueba si una fecha introducida por el usuario es válida. Para ello, divide la fecha en año, mes y día mediante split("-"), convierte esas partes a números y utiliza new Date() para comprobar que la fecha existe realmente.
  - correoValido: comprueba que el correo tenga una estructura válida mediante una expresión regular.
  - telefonoValido: comprueba que el teléfono tenga exactamente 9 dígitos, también mediante una expresión regular.
  - imagenValida: comprueba que el nombre de la imagen termine en .jpg

    - mostrarErrores: esta función recibe el evento del formulario y un array con los mensajes de error. Si el array contiene algún error, se detiene el envío del formulario mediante evento.preventDefault() y se muestran todos los errores juntos mediante un alert.

- validacionLogin.js

  Este fichero se encarga de validar el formulario de inicio de sesión que aparece en la cabecera de la web. Primero localiza el formulario mediante su identificador y después comprueba que el formulario exista en la página. Cuando el usuario intenta enviar el formulario, se recogen los valores de los campos de usuario y contraseña.
  A continuación, se comprueba que ninguno de los dos campos esté vacío. Si falta alguno, se añade un mensaje al array de errores. Finalmente, se llama a mostrarErrores para detener el envío si existen errores.

- validacionRegistro.js

  El fichero validacionRegistro.js valida el formulario de registro de usuarios de la página altausuarios.php. Este archivo comprueba los campos introducidos por el usuario antes de enviar el formulario a actions/procesarAltaUsuario.php.

  Para comprobar el nombre y el usuario se utiliza la función estaVacio(), asegurando que los campos no sean nulos.

  La fecha de nacimiento se valida mediante fechaValida(), que comprueba que la fecha tenga una estructura correcta y que sea una fecha real.

  El teléfono se valida mediante telefonoValido(), asegurando que tenga 9 dígitos y, el correo mediante correoValido().

  También se valida que el campo de contraseña no esté vacío y que la confirmación de contraseña coincida con la contraseña introducida. Si ambas contraseñas no coinciden, se muestra un mensaje de error.

- validacionBuscador.js

  El fichero validacionBuscador.js valida el formulario del buscador de viajes situado en la página principal. Este buscador permite realizar búsquedas por destino, por fecha o por ambos. Por ello, la validación comprueba que no estén vacío los dos campos al mismo tiempo.

  Además, se el usuario introduce una fecha, se comprueba que sea válida mediante la función fechaValida().

- validacionViaje.js

  El fichero validacionViaje.js se utiliza en los formularios de creación y edición de viajes dentro de gestion_viajes.php. Este archivo valida todos los datos necesarios para crear o modificar un viaje.

  Los campos obligatorios se validan con estaVacio() y las fechas se validan con fechaValida(). Además, se comprueba que la fecha de fin sea posterior a la fecha de inicio.

  La duración y el precio se validan mediante numeroPositivo(), asegurando que sean valores mayores que cero. La imagen se valida con imagenValida(), comprobando que el archivo termine .jpg.

4. USO DE INTELIGENCIA ARTIFICIAL

En esta práctica he usado herramientas de inteligencia artificial, como ChatGPT, para resolver algunas dudas, revisar errores y mejorar la organización del código.

En primer lugar, se ha utilizado IA para resolver problemas relacionados con la sintaxis de PHP y dudas sobre el uso de session_start(), require_once, redirecciones con header() y consultas preparadas.

También se ha empleado para solucionar errores relacionados con el cifrado de contraseñas, ya que al incorporar password_hash() y password_verify(), fue necesario modificar la inserción de usuario y el sistema de login.

Para terminar la parte de PHP, la IA se ha utilizado para detectar partes de código repetidas y refactorizar esa lógica en funciones reutilizables, reduciendo la repetición de código.

En cuanto al uso de IA para la parte de JavaScript, en estos ficheros se ha utilizado para comprender el funcionamiento y la definición de las expresiones regulares en JS, ya que algunas validaciones, como el formato del correo electrónico o las fechas, requerían comprobar patrones concretos.

Por último, también ha ayudado a resolver problemas del carrusel dinámico de la página index.php. Inicialmente, el carrusel estaba formado por imágenes fijas, pero gracia a la IA se ha conseguido pasar datos de PHP a JavaScript mediante json_encode(), recorrer el array de viajes y controlar las flechas de navegación del carrusel.

5. ELEMENTOS DE INNOVACIÓN

Como elemento de innovación se ha incorporado el cifrado de contraseñas en el sistema de registro e inicio de sesión de usuarios. De esta forma, se mejora la seguridad y se evita que las contraseñas se almacenen en texto plano dentro de la base de datos.

Cuando un usuario se registra, la contraseña introducida en el formulario se cifra antes de insertar en la base de datos. De esta forma, en la tabla “usuario” no se guarda la contraseña original, sino una cadena cifrada.

Cuando un usuario quiere iniciar sesión se busca mediante una consulta SQL el usuario y se comprueba la contraseña mediante password_verify(). Con esta comprobación, PHP compara la contraseña escrita por el usuario con la contraseña cifrada en la base de datos. Si ambas coinciden, se inicia la sesión.

6. REFERENCIAS CONSULTADAS

- https://www.php.net/manual/en/function.password-hash.php
- https://www.php.net/manual/en/function.password-verify.php
- Guiones de prácticas aportados para el aprendizaje de PHP
- Guiones de prácticas aportados para el aprendizaje de JavaScript